9.2 Macro Definition and Substitution

[This section corresponds to K&R Sec. 4.11.2]

A preprocessor line of the form

```
#define name text
```

defines a *macro* with the given name, having as its *value* the given replacement text. After that (for the rest of the current source file), wherever the preprocessor sees that name, it will replace it with the replacement text. The name follows the same rules as ordinary identifiers (it can contain only letters, digits, and underscores, and may not begin with a digit). Since macros behave quite differently from normal variables (or functions), it is customary to give them names which are all capital letters (or at least which begin with a capital letter). The replacement text can be absolutely anything--it's not restricted to numbers, or simple strings, or anything.

The most common use for macros is to propagate various constants around and to make them more self-documenting. We've been saying things like

```
char line[100];
...
getline(line, 100);
```

but this is neither readable nor reliable; it's not necessarily obvious what all those 100's scattered around the program are, and if we ever decide that 100 is too small for the size of the array to hold lines, we'll have to remember to change the number in two (or more) places. A much better solution is to use a macro:

```
#define MAXLINE 100
char line[MAXLINE];
...
getline(line, MAXLINE);
```

Now, if we ever want to change the size, we only have to do it in one place, and it's more obvious what the words `MAXLINE` sprinkled through the program mean than the magic numbers 100 did.

Since the replacement text of a preprocessor macro can be anything, it can also be an expression, although you have to realize that, as always, the text is substituted (and perhaps evaluated) later. No evaluation is performed when the macro is defined. For example, suppose that you write something like

```
#define A 2
#define B 3
#define C A + B
```

(this is a pretty meaningless example, but the situation does come up in practice). Then, later, suppose that you write

```
int x = C * 2;
```

If `A`, `B`, and `C` were ordinary variables, you'd expect `x` to end up with the value 10. But let's see what happens.

The preprocessor always substitutes text for macros exactly as you have written it. So it first substitites the replacement text for the macro `C`, resulting in

```
int x = A + B * 2;
```

Then it substitutes the macros `A` and `B`, resulting in

```
int x = 2 + 3 * 2;
```

Only when the preprocessor is done doing all this substituting does the compiler get into the act. But when it evaluates that expression (using the normal precedence of multiplication over addition), it ends up initializing `x` with the value 8!

To guard against this sort of problem, it is always a good idea to include explicit parentheses in the definitions of macros which contain expressions. If we were to define the macro `C` as

```
#define C (A + B)
```

then the declaration of `x` would ultimately expand to

```
int x = (2 + 3) * 2;
```

and `x` would be initialized to 10, as we probably expected.

Notice that there does not have to be (and in fact there usually is *not*) a semicolon at the end of a `#define` line. (This is just one of the ways that the syntax of the preprocessor is different from the rest of C.) If you accidentally type

```
#define MAXLINE 100;                    /* WRONG */
```

then when you later declare

```
char line[MAXLINE];
```

the preprocessor will expand it to

```
char line[100;];                        /* WRONG */
```

which is a syntax error. This is what we mean when we say that the preprocessor doesn't know much of anything about the syntax of C--in this last example, the *value* or replacement text for the macro `MAXLINE` was the 4 characters `1 0 0 ;` , and that's exactly what the preprocessor substituted (even though it didn't make any sense).

Simple macros like `MAXLINE` act sort of like little variables, whose values are constant (or constant expressions). It's also possible to have macros which look like little functions (that is, you invoke them with what looks like function call syntax, and they expand to replacement text which is a function of the actual arguments they are invoked with) but we won't be looking at these yet.

Read sequentially: prev next up top